

Name - Sakshi Shingole

Roll no - 150096724056

Cohort - Jensen Huang

Draw io link -

https://drive.google.com/file/d/1LnDKkTtasZ8ihcgObiYlbP9c_JSkuHkv/view?usp=sharing

1. Problem Statement

Notification Management System

Problem Statement

Modern applications need an efficient way to deliver notifications to users through multiple communication channels such as Email, SMS, and Push Notifications. Managing these notifications manually is inefficient and difficult to scale.

The objective of this project is to design and implement a Notification Management System that receives notification requests, validates them, processes them asynchronously through a notification queue, routes them to the appropriate notification service, updates the delivery status, and stores the complete notification history in a database.

2. Proposed Solution

The proposed Notification Management System uses an event-driven architecture where notification requests are validated before being stored in a SQLite database. Each notification is placed into a queue and processed by a background worker. The Channel Router identifies the selected notification channel and forwards the request to the appropriate notification service (Email, SMS, or Push Notification). Once the notification is delivered, its status is updated in the database and the event is recorded by the Logger.

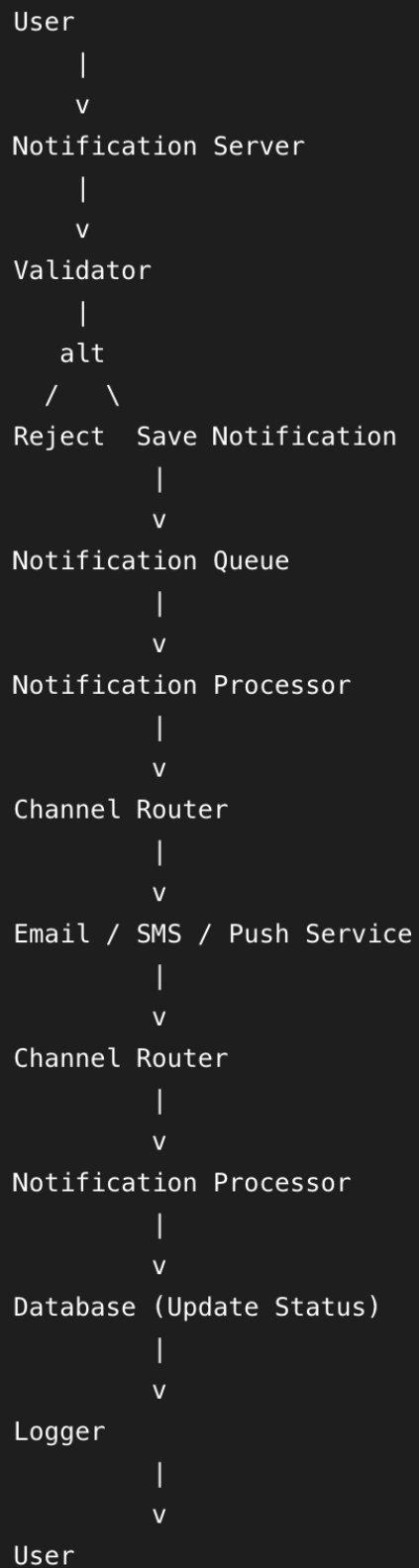
This modular architecture improves maintainability, scalability, and separation of responsibilities while keeping the implementation simple and easy to understand.

3. Objectives

- Design a modular Notification Management System.
- Support multiple notification channels.

- Validate notification requests before processing.
- Store notification details in a database.
- Process notifications asynchronously using a queue.
- Maintain delivery history.
- Demonstrate event-driven system design concepts.

4. System Architecture

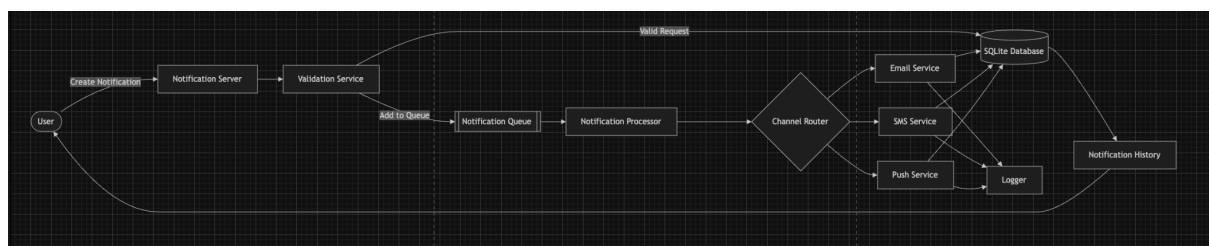


Description

The system consists of several independent components that communicate to process notification requests.

1. **User**
 - Initiates notification requests.
2. **Notification Server**
 - Receives notification requests.
3. **Validation Service**
 - Validates notification details.
4. **Notification Queue**
 - Stores pending notification requests.
5. **Notification Processor**
 - Retrieves notifications from the queue.
6. **Channel Router**
 - Selects the appropriate notification channel.
7. **Notification Services**
 - Email Service
 - SMS Service
 - Push Notification Service
8. **SQLite Database**
 - Stores notifications and their status.
9. **Logger**
 - Records notification processing events.
10. **Notification History**
 - Displays processed notifications.

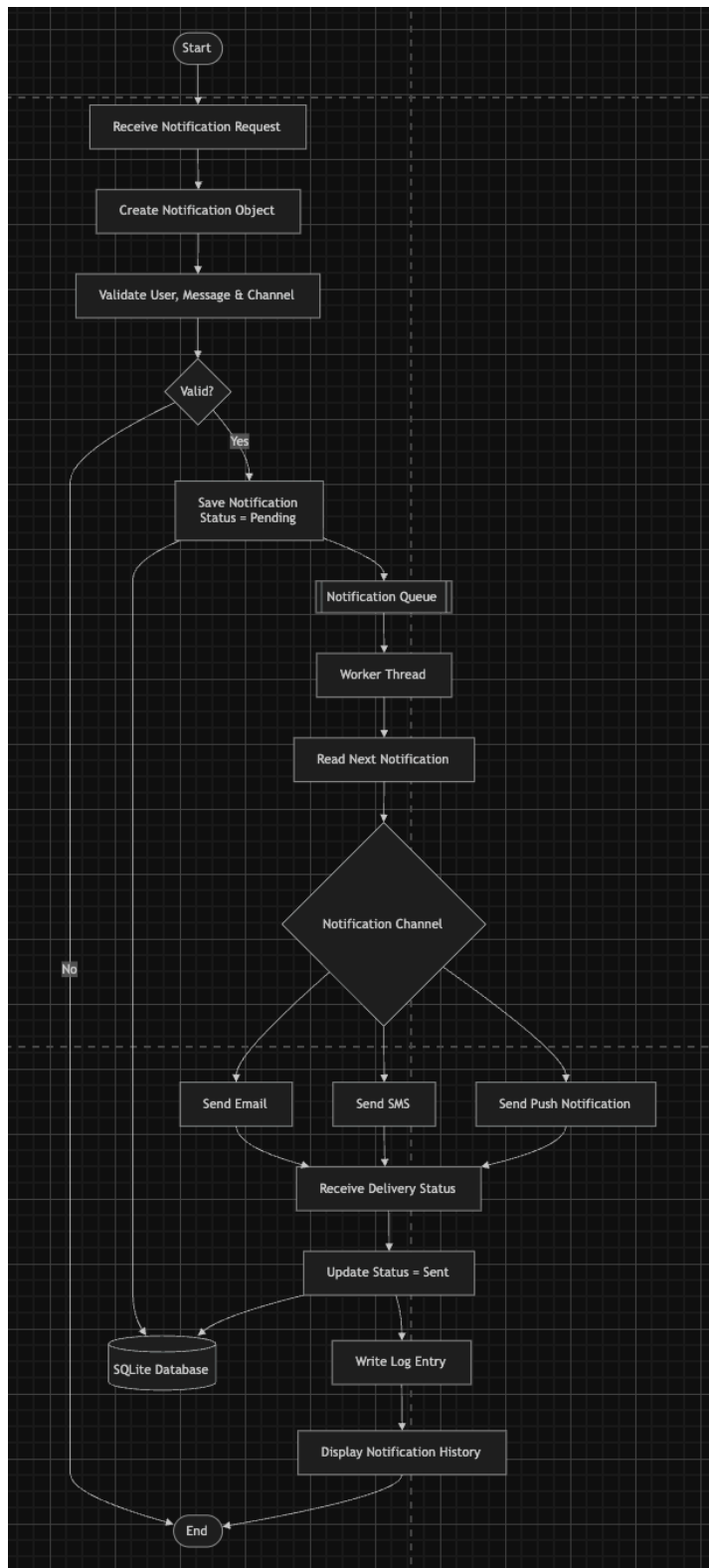
5. High Level Design (HLD)



Description

The High-Level Design illustrates the major components of the Notification Management System and their interactions. It shows how notification requests flow from the user through validation, queuing, processing, routing, notification delivery, database storage, and logging.

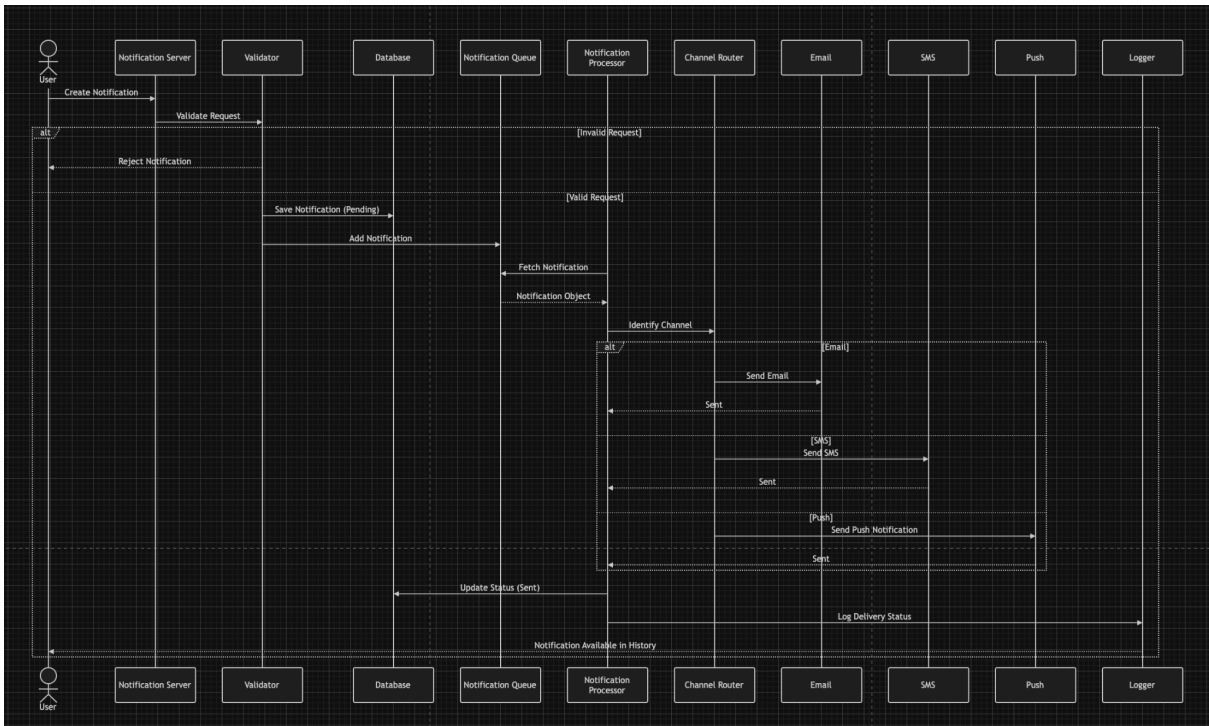
6. Low Level Design (LLD)



Description

The Low-Level Design provides a detailed representation of the internal processing flow. It illustrates request validation, queue operations, notification processing, channel routing, status updates, and database interactions.

7. Event Flow Diagram



Description

The Event Flow Diagram describes the chronological interaction between different system components during the lifecycle of a notification request, from creation to successful delivery and logging.

8. Module Description

Notification Server

Responsibilities

- Accept notification requests.
- Create notification objects.
- Validate user input.

- Store pending notifications.
- Add notifications to the queue.

Validation Service

Responsibilities

- Validate user name.
- Validate notification message.
- Validate notification channel.

Notification Queue

Responsibilities

- Temporarily store notifications.
- Process notifications in FIFO order.

Notification Processor

Responsibilities

- Retrieve notifications from the queue.
- Route notifications to the correct service.
- Update delivery status.
- Record processing events.

Channel Router

Responsibilities

- Identify the selected notification channel.
- Forward requests to the appropriate notification service.

Email Service

Responsibilities

- Simulate email delivery.
- Return delivery status.

SMS Service

Responsibilities

- Simulate SMS delivery.
- Return delivery status.

Push Notification Service

Responsibilities

- Simulate push notification delivery.
- Return delivery status.

SQLite Database

Responsibilities

- Store notification records.
- Update notification status.
- Retrieve notification history.

Logger

Responsibilities

- Record processing events.
- Display notification activity.

9. Database Design

Table: notifications

Column	Data Type	Description
notification_id	TEXT	Unique notification identifier
username	TEXT	Recipient name
channel	TEXT	Notification channel
message	TEXT	Notification content
status	TEXT	Delivery status
created_at	TEXT	Notification timestamp

Database Explanation

The SQLite database stores all notification records. Each notification is uniquely identified by a Notification ID and contains information about the recipient, selected channel, message, processing status, and creation timestamp.

10. Technology Stack

Layer	Technology
Programming Language	Python
Notebook Environment	Google Colab / Jupyter Notebook
Database	SQLite
Queue	Python queue.Queue
Threading	Python threading
Frontend Simulation	HTML, CSS
Documentation	Mermaid
Version Control	Git & GitHub

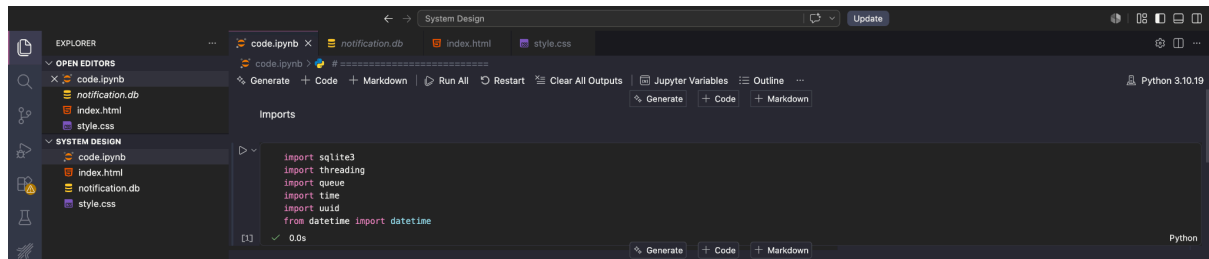
11. Implementation Details

Step 1: Import Required Libraries

Description:

The required Python libraries such as sqlite3, threading, queue, time, uuid, and datetime are imported. These libraries provide database connectivity, multithreading, queue management, unique ID generation, and timestamp handling.

Screenshot: Imports cell.



Step 2: Database Initialization

Description:

A SQLite database (notification.db) is created, and the notifications table is initialized to store notification details such as Notification ID, Username, Channel, Message, Status, and Timestamp. Functions for inserting, updating, and displaying notifications are also implemented.

Screenshot: Database initialization + save_notification(), update_status(), and show_notifications() functions.

The screenshot shows a Jupyter Notebook in VS Code with the following code:

```
Imports

import sqlite3
import threading
import queue
import time
import uuid
from datetime import datetime

[1] ✓ 0.0s Python

Database

# =====
# DATABASE
# =====

conn = sqlite3.connect("notification.db", check_same_thread=False)
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS notifications(
    notification_id TEXT PRIMARY KEY,
    username TEXT,
    channel TEXT,
    message TEXT,
    status TEXT,
    created_at TEXT
)
""")

conn.commit()

[2] ✓ 0.0s Python

#save notification to database

def save_notification(notification):

    cursor.execute("""
INSERT INTO notifications
VALUES(?, ?, ?, ?, ?)
""", (
    notification["id"],
    notification["user"],
    notification["channel"],
    notification["message"],
    notification["status"],
    notification["time"]
))

    conn.commit()

[3] ✓ 0.0s Python
```

The screenshot shows a Jupyter Notebook in VS Code with the following code:

```
INSERT INTO notifications
VALUES(?, ?, ?, ?, ?)
""", (
    notification["id"],
    notification["user"],
    notification["channel"],
    notification["message"],
    notification["status"],
    notification["time"]
))

    conn.commit()

[3] ✓ 0.0s Python

#Update Status

def update_status(notification_id, status):

    cursor.execute("""
UPDATE notifications
SET status=?
WHERE notification_id=?
""", (status, notification_id))

    conn.commit()

[4] ✓ 0.0s Python

#Display Database

def show_notifications():

    cursor.execute("SELECT * FROM notifications")

    rows=cursor.fetchall()

    print("\nNotification History\n")

    for row in rows:

        print(row)

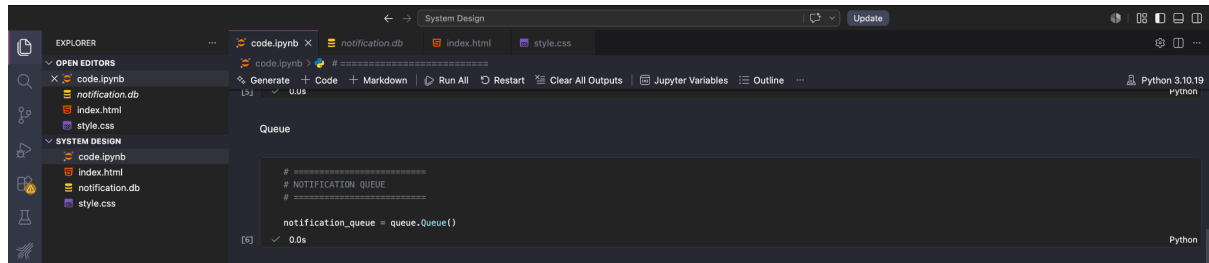
[5] ✓ 0.0s Python
```

Step 3: Queue Initialization

Description:

A FIFO notification queue is created using Python's `queue.Queue()`. The queue temporarily stores notification requests until they are processed by the worker thread.

Screenshot: Queue cell.



```
# =====  
# NOTIFICATION QUEUE  
# =====  
  
notification_queue = queue.Queue()
```

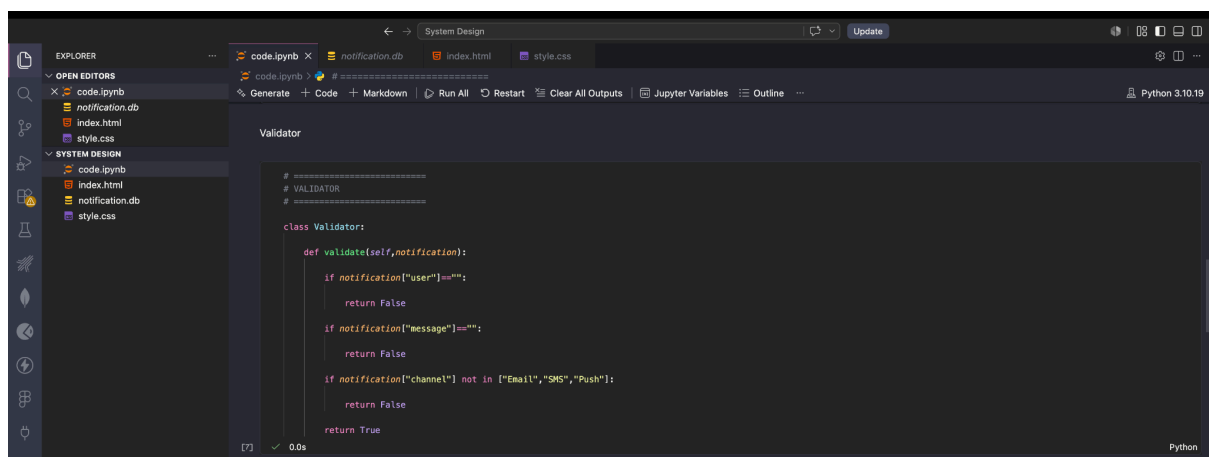
The screenshot shows a Jupyter Notebook interface with a file explorer on the left containing files like `code.ipynb`, `notification.db`, `index.html`, and `style.css`. The main editor area displays a code cell titled "Queue" with the above code. The code defines a notification queue using `queue.Queue()`. The output of the cell is `[5]` and `0.0s`.

Step 4: Notification Validation

Description:

The Validator class validates each notification request by checking whether the username and message are provided and whether the selected notification channel is one of the supported channels (Email, SMS, or Push). Invalid requests are rejected before entering the queue.

Screenshot: Validator class.



```
# =====  
# VALIDATOR  
# =====  
  
class Validator:  
  
    def validate(self,notification):  
  
        if notification["user"]=="":  
            return False  
  
        if notification["message"]=="":  
            return False  
  
        if notification["channel"] not in ["Email","SMS","Push"]:  
            return False  
  
        return True
```

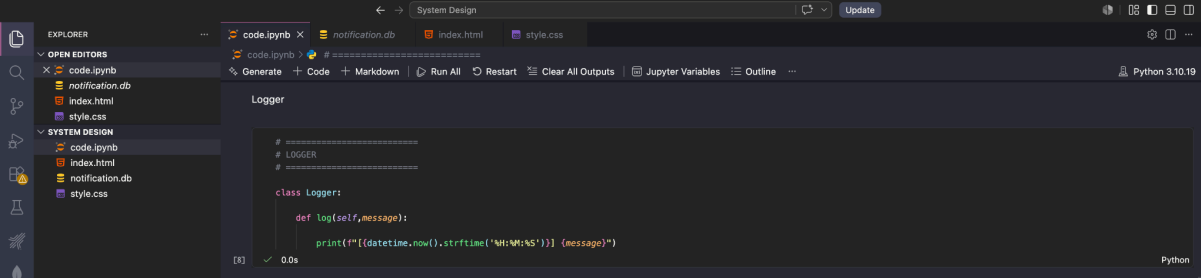
The screenshot shows a Jupyter Notebook interface with a file explorer on the left containing files like `code.ipynb`, `notification.db`, `index.html`, and `style.css`. The main editor area displays a code cell titled "Validator" with the above code. The code defines a `Validator` class with a `validate` method that checks if the `user`, `message`, and `channel` fields of a notification are valid. The output of the cell is `[7]` and `0.0s`.

Step 5: Logging Mechanism

Description:

The Logger class records important events such as notification submission, processing, and delivery status. These log messages help monitor the execution of the notification system.

Screenshot: Logger class.

A screenshot of a code editor interface. The Explorer panel on the left shows a project structure with files: code.ipynb, notification.db, index.html, and style.css. The main editor area displays the code for the 'Logger' class. The code is as follows:

```
# =====  
# LOGGER  
# =====  
  
class Logger:  
  
    def log(self,message):  
        printf("[%datetime.now().strftime('%H:%M:%S')]"] {message}")
```

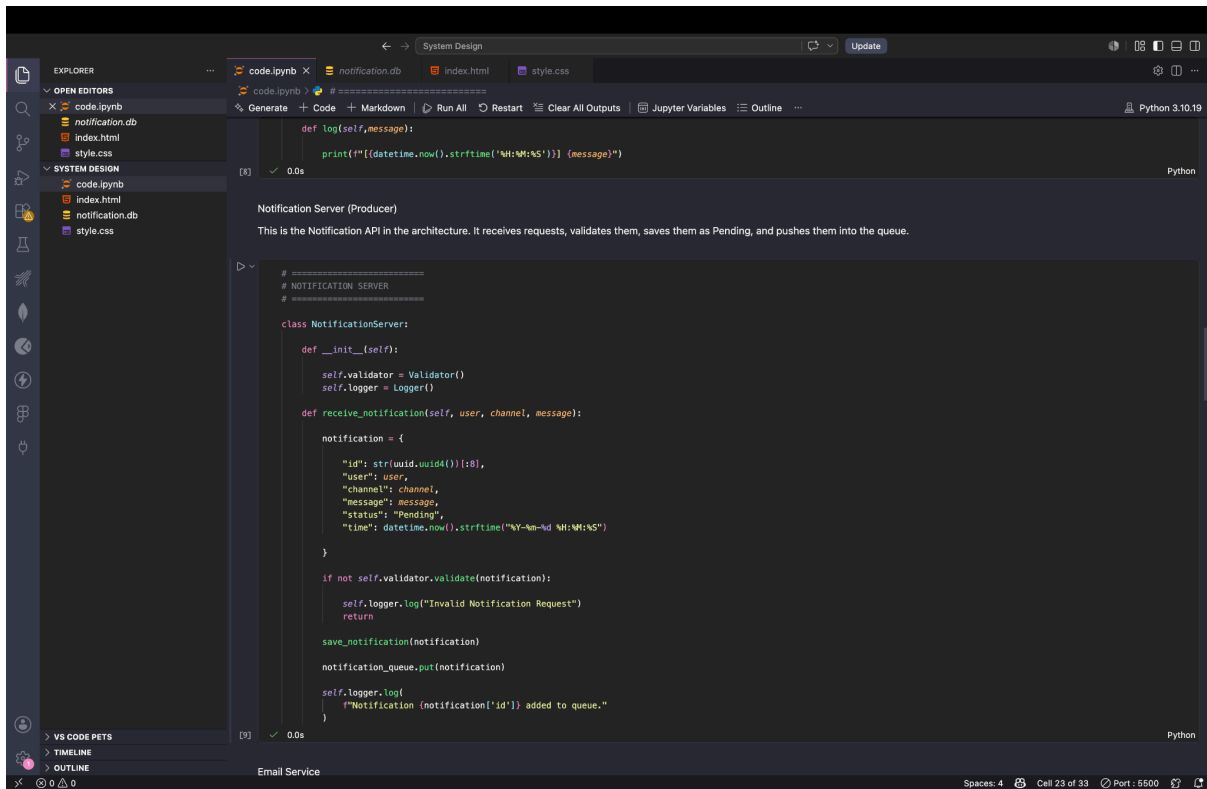
The editor also shows a toolbar with options like Generate, Code, Markdown, Run All, Restart, and Clear All Outputs. The bottom right corner indicates the Python version is 3.10.19.

Step 6: Notification Server (Producer)

Description:

The NotificationServer acts as the producer in the system. It receives notification requests, generates a unique notification ID, validates the request, stores it in the database with **Pending** status, and places it into the notification queue.

Screenshot: NotificationServer class.



Step 7: Notification Delivery Services

Description:

Separate service classes simulate the delivery of notifications through Email, SMS, and Push Notification channels. Each service prints the delivery message and returns the status as **Sent** after processing.

Screenshot: EmailService, SMSService, and PushService classes.

The screenshot shows the VS Code System Design editor with three service components defined in a code file:

- Email Service** (lines 10-14):

```
# =====  
# EMAIL SERVICE  
# =====  
  
class EmailService:  
    def send(self, notification):  
        print(f"Sending Email to {notification['user']}")  
        time.sleep(2)  
        return "Sent"
```
- SMS Service** (lines 17-21):

```
# =====  
# SMS SERVICE  
# =====  
  
class SMSService:  
    def send(self, notification):  
        print(f"Sending SMS to {notification['user']}")  
        time.sleep(2)  
        return "Sent"
```
- Push Notification Service** (lines 24-28):

```
# =====  
# PUSH SERVICE  
# =====  
  
class PushService:  
    def send(self, notification):
```

The Explorer sidebar on the left shows the project structure: code.ipynb, notification.db, index.html, and style.css. The bottom status bar indicates "Spaces: 4", "Cell 23 of 33", and "Port: 5500".

This screenshot continues the code from the previous one, showing the **Channel Router** service (lines 31-35):

```
# =====  
# PUSH SERVICE  
# =====  
  
class PushService:  
    def send(self, notification):  
        print(f"Sending Push Notification to {notification['user']}")  
        time.sleep(2)  
        return "Sent"
```

Below this, the **Channel Router** service is partially visible:

```
class ChannelRouter:
```

The Explorer sidebar and status bar are consistent with the previous screenshot.

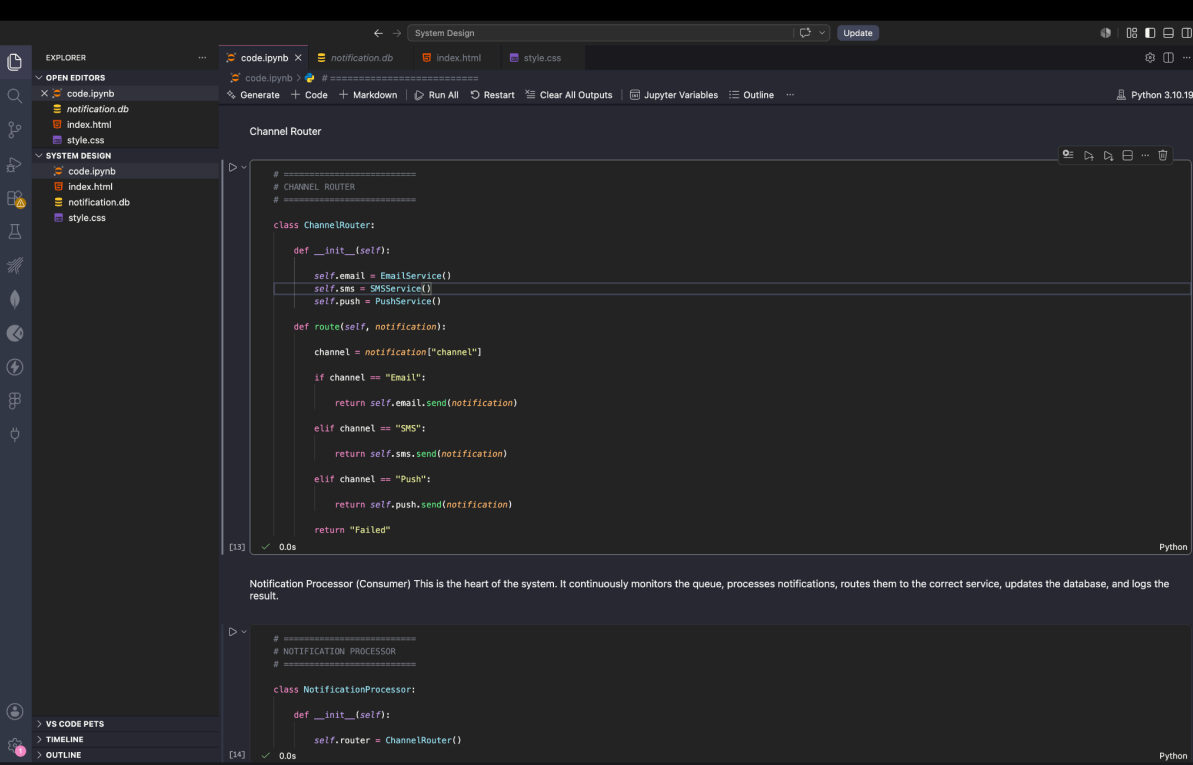
Step 8: Channel Router

Description:

The ChannelRouter class determines the selected notification channel

and forwards the notification to the corresponding delivery service (Email, SMS, or Push).

Screenshot: ChannelRouter class.



The screenshot shows a VS Code editor with a dark theme. The Explorer sidebar on the left shows a project named 'SYSTEM DESIGN' with files 'code.ipynb', 'index.html', 'notification.db', and 'style.css'. The main editor area displays two Python code files. The top file, 'code.ipynb', contains the 'ChannelRouter' class. The bottom file, 'notification.db', contains the 'NotificationProcessor' class. The status bar at the bottom indicates 'Python 3.10.19' and 'Cell 23 of 33'.

```
# =====  
# CHANNEL ROUTER  
# =====  
  
class ChannelRouter:  
    def __init__(self):  
        self.email = EmailService()  
        self.sms = SMSService()  
        self.push = PushService()  
  
    def route(self, notification):  
        channel = notification["channel"]  
  
        if channel == "Email":  
            return self.email.send(notification)  
  
        elif channel == "SMS":  
            return self.sms.send(notification)  
  
        elif channel == "Push":  
            return self.push.send(notification)  
  
        return "Failed"
```

Notification Processor (Consumer) This is the heart of the system. It continuously monitors the queue, processes notifications, routes them to the correct service, updates the database, and logs the result.

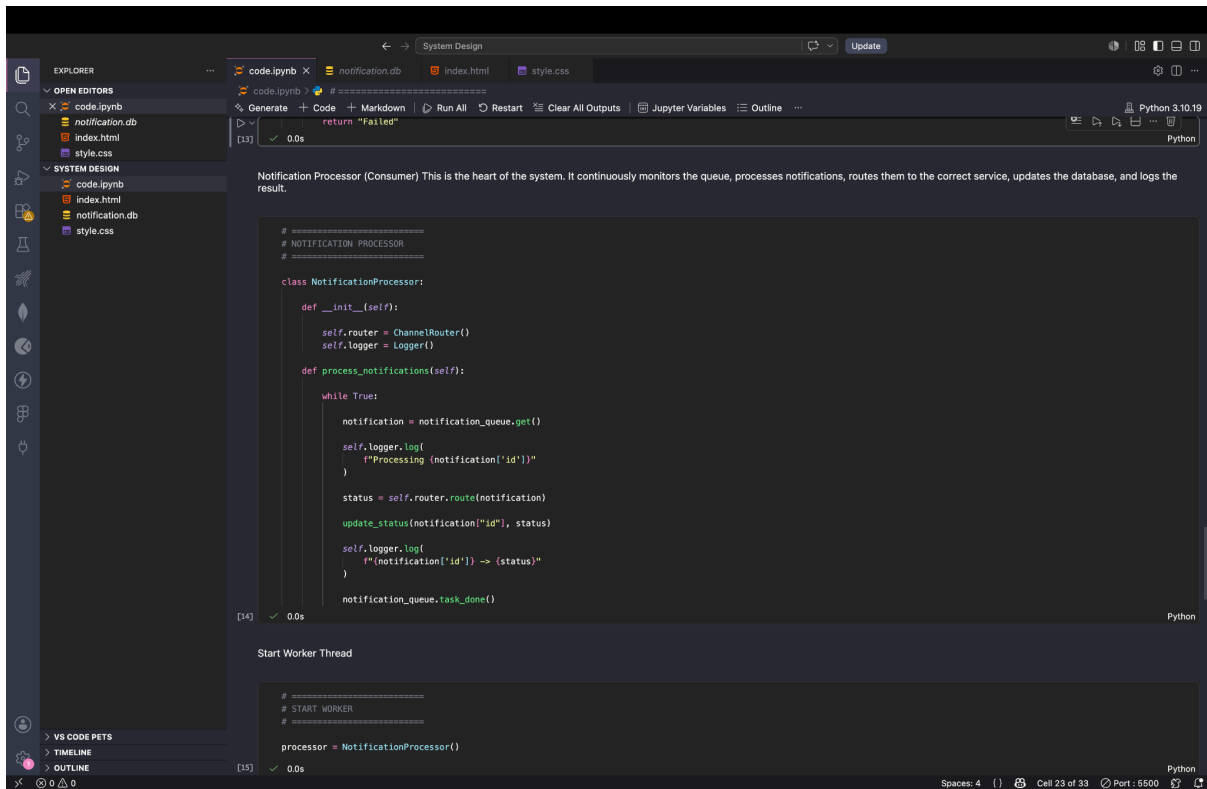
```
# =====  
# NOTIFICATION PROCESSOR  
# =====  
  
class NotificationProcessor:  
    def __init__(self):  
        self.router = ChannelRouter()
```

Step 9: Notification Processor (Consumer)

Description:

The NotificationProcessor continuously monitors the notification queue using a worker thread. It retrieves queued notifications, routes them through the appropriate delivery service, updates their status in the database, logs the processing details, and marks the task as completed.

Screenshot: NotificationProcessor class.

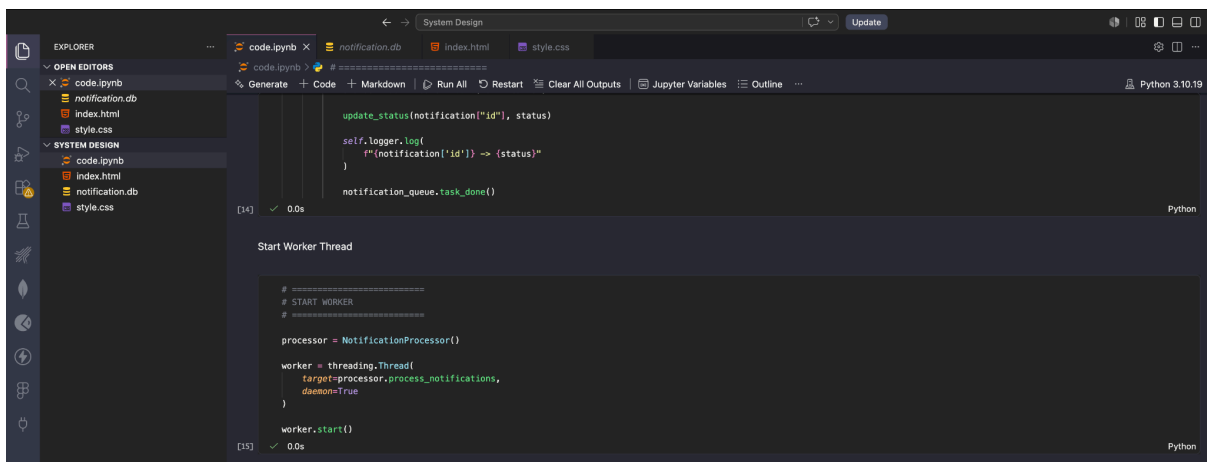


Step 10: Worker Thread Initialization

Description:

A background worker thread is created to execute the notification processor. This allows notification processing to occur asynchronously while new notifications continue to enter the queue.

Screenshot: Start Worker Thread cell.

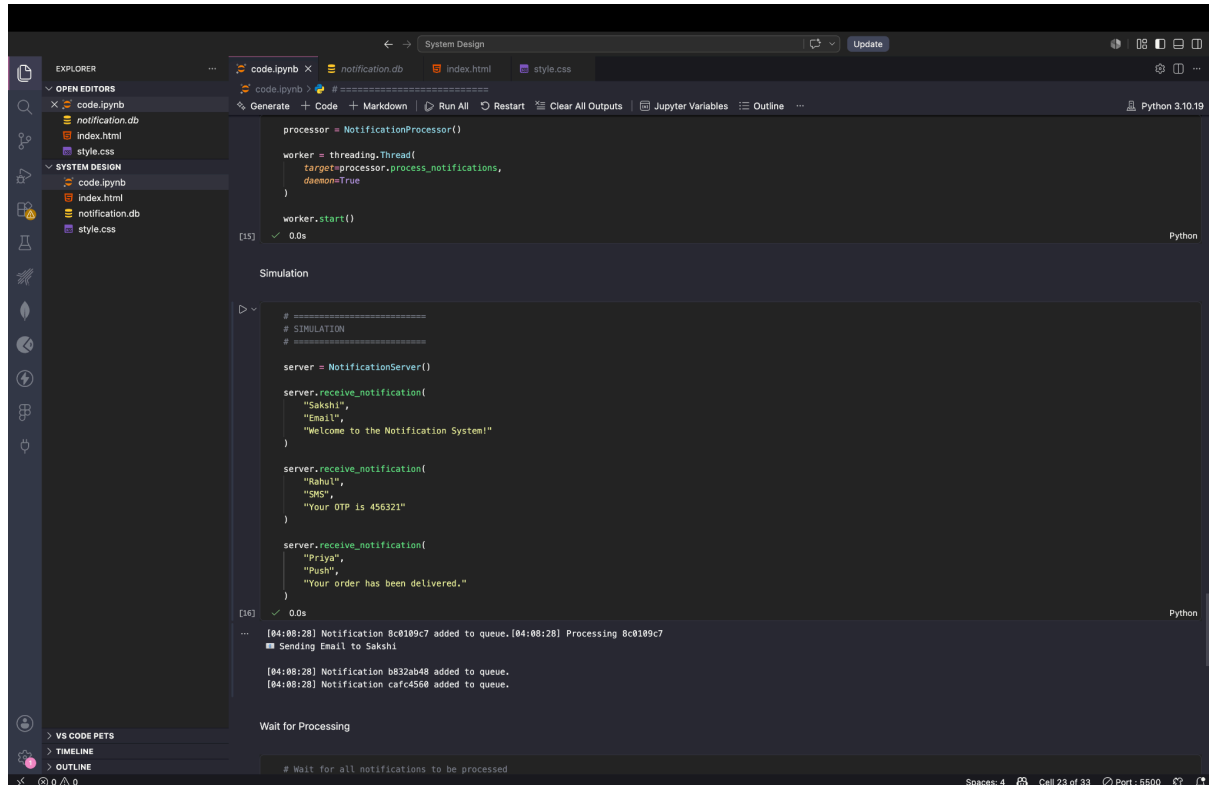


Step 11: Simulation Execution

Description:

Sample notification requests are submitted to the Notification Server using different communication channels (Email, SMS, and Push). This demonstrates the end-to-end workflow of the notification system.

Screenshot: Simulation cell.



```
processor = NotificationProcessor()

worker = threading.Thread(
    target=processor.process_notifications,
    daemon=True
)

worker.start()

# SIMULATION
# =====
server = NotificationServer()

server.receive_notification(
    "Sakshi",
    "Email",
    "Welcome to the Notification System!"
)

server.receive_notification(
    "Rahul",
    "SMS",
    "Your OTP is 456321"
)

server.receive_notification(
    "Priya",
    "Push",
    "Your order has been delivered."
)

[04:08:28] Notification 8c0109c7 added to queue. [04:08:28] Processing 8c0109c7
[04:08:28] Notification b832ab48 added to queue.
[04:08:28] Notification cafc4568 added to queue.

Wait for Processing

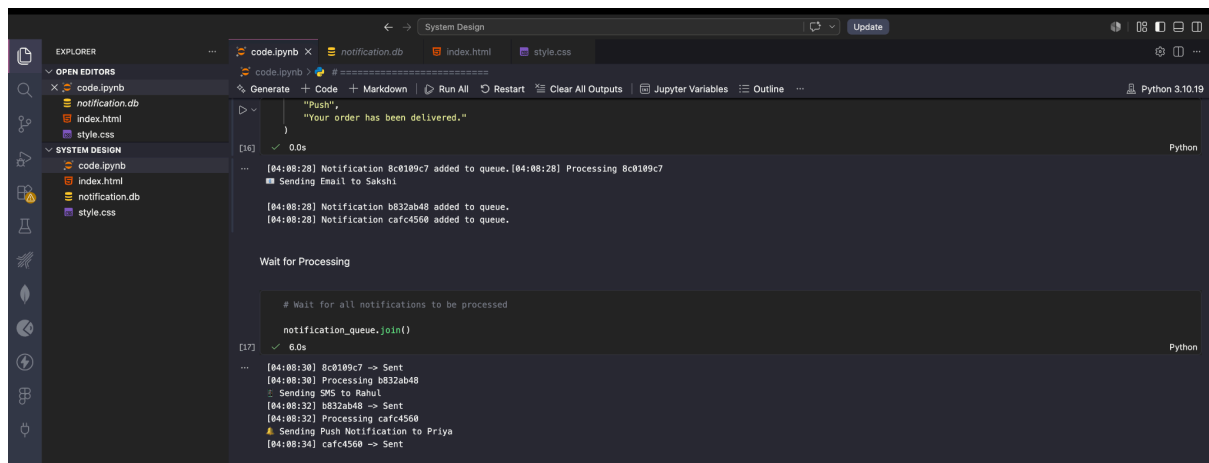
# Wait for all notifications to be processed
```

Step 12: Queue Processing

Description:

The program waits until all queued notifications have been processed using `notification_queue.join()`. This ensures that every notification has been successfully delivered before terminating the execution.

Screenshot: Wait for Processing output.



The screenshot shows a Jupyter Notebook interface with a dark theme. The Explorer panel on the left shows a file structure with 'code.ipynb', 'notification.db', 'index.html', and 'style.css'. The main editor area displays the notebook content, which includes a code cell with a 'Wait for Processing' section. The output of the notebook shows a sequence of events: a notification is added to the queue, processed, and then sent via email, SMS, and push notification. The output is as follows:

```
[04:08:28] Notification 8c0109c7 added to queue. [04:08:28] Processing 8c0109c7
[04:08:28] Sending Email to Sakshi

[04:08:28] Notification b832ab48 added to queue.
[04:08:28] Notification cafc4560 added to queue.

Wait for Processing

# Wait for all notifications to be processed
notification_queue.join()

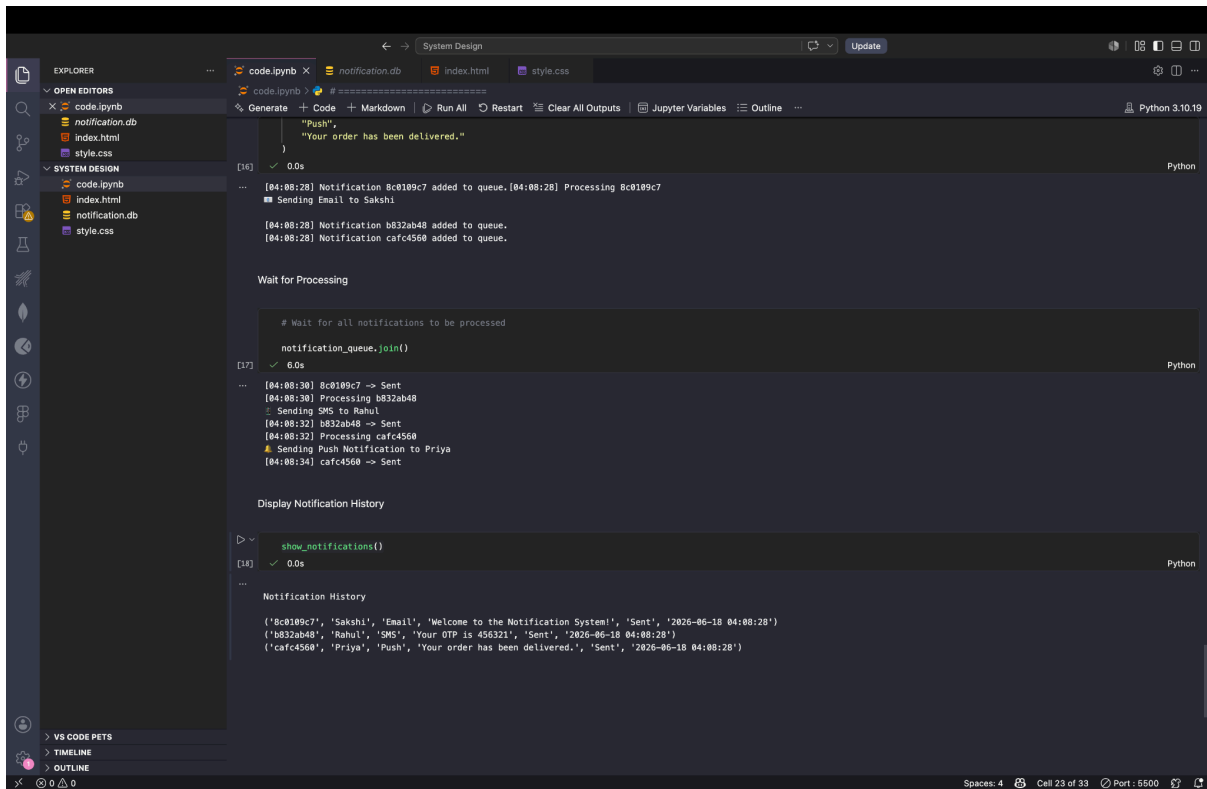
[04:08:30] 8c0109c7 -> Sent
[04:08:30] Processing b832ab48
[04:08:30] Sending SMS to Rahul
[04:08:32] b832ab48 -> Sent
[04:08:32] Processing cafc4560
[04:08:32] Sending Push Notification to Priya
[04:08:34] cafc4560 -> Sent
```

Step 13: Display Notification History

Description:

Finally, all notification records stored in the SQLite database are retrieved and displayed. The output includes the notification ID, username, channel, message, delivery status, and timestamp, confirming successful processing.

Screenshot: Notification History output.



12. Project Structure

NotificationManagementSystem.ipynb

- Imports
- Database
- Queue
- Validator
- Logger
- Notification Server
- Email Service
- SMS Service
- Push Service
- Channel Router
- Notification Processor
- Simulation

13. Results & Screenshots

This Includes screenshots of:

1. Notebook execution
2. Notification simulation output
3. SQLite database records
4. HTML simulation page
5. HLD diagram
6. LLD diagram
7. Event Flow diagram
8. GitHub repository

14. Future Scope

- Integrate real Email APIs such as SMTP or SendGrid.
- Integrate SMS gateways such as Twilio.
- Integrate Firebase Cloud Messaging (FCM) for push notifications.
- Add user authentication and authorization.
- Support scheduled notifications.
- Implement retry mechanisms for failed notifications.
- Add a web dashboard for monitoring notification status and analytics.